

TECHNICAL DOCUMENTATION

Planet Render 360

Render any point on the map as a geographically faithful 3D, 360-degree view — built on device from free public data, in Godot.

ENGINE	TARGETS	EDITION	LICENSE
Godot 4.7 · GDScript	macOS · iOS · Android	November 2026	MIT

AUTHOR

Petri Tuononen | https://www.linkedin.com/in/petrituononen

CONTENTS

What is inside

This document describes how Planet Render 360 turns a latitude and longitude into a walkable-looking panorama: the architecture, the data it fetches, the staged generation pipeline, the world model, rendering, caching and verification.

01	**Overview & principles**	Goals, hard constraints, layered architecture	3
02	**Screens & user flow**	Screen manager, state transitions, UI	4
03	**Data sources**	Twelve free public sources, register gate	5
04	**Generation pipeline**	Four stages, overlapped fetches, fallbacks	6
05	**World model**	LocationData, WorldScale, GroundTruth	7
06	**World assembly**	Parallel build, buildings, vegetation	8
07	**LOD & rendering**	Static LOD rings, atmosphere, sun	9
08	**Quality presets & governor**	Fast → Ultra, FPS-driven stepping	10
09	**Caching & offline**	Data cache, built-world cache, history	11
10	**Landmarks**	Registry, mapped OSM, archetypes, Wikipedia	12
11	**Code organisation & testing**	Packages, size budget, test suite	13
12	**Gallery & attribution**	Captures, licences, credits	14

At a glance

209 GDSCRIPT CLASSES	45.9k LINES OF CODE	52 TEST SCRIPTS	12 FREE DATA SOURCES
4 PIPELINE STAGES	2.4 km DETAILED EXTENT	33 km DEM HORIZON	23 BESPOKE LANDMARKS

Source of truth: this PDF summarises the code and the living documents in `docs/` (world-model contract, data-source register, fidelity scorecard) and `.agents/plans/`. Where they differ, the code and the contract win.

Overview & principles

Pick a place by tapping a world map, searching, or typing coordinates. The app fetches real elevation, imagery, map features and climate for that point, builds a 3D world in true metres around it, and drops you in the middle with a drag-to-look 360° camera.

Hard constraints

- **No self-hosted server.** The device talks directly to free public sources; no proxy, account or backend.
- **Free, no paywall.** No purchases, subscriptions or locked tiers; no paid API keys.
- **Mobile-first.** Smooth on a mid-range phone; desktop may be prettier, mobile budgets win.
- **Few-minute loading.** Progress is real per-stage work, never a fake timer.

Design invariants

- **One unit is one metre** — horizontally and vertically, end to end.
- **Real data first.** Procedural rules run only on *absence* of real data for a layer, never on top of it.
- **Data contracts before visuals.** Generation never parses provider JSON or PNG; fetchers normalise into `LocationData`.
- **Attribution is UI.** Credits shown in-app come from the fetchers actually used.

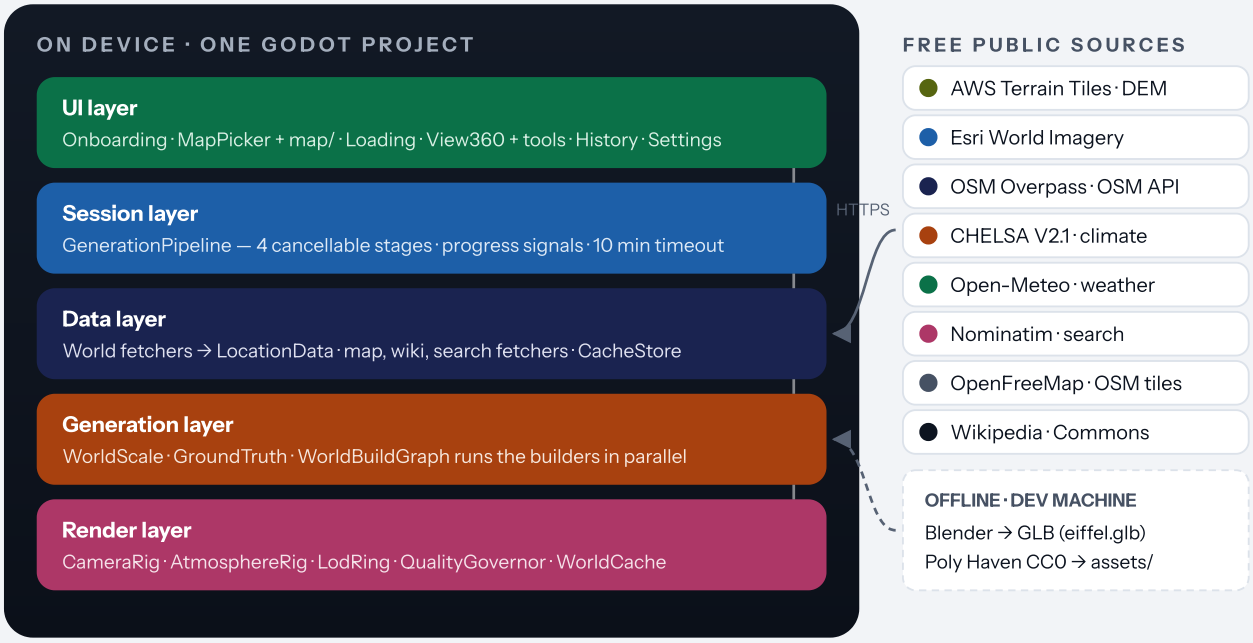


Figure 1. Layered architecture. Every layer lives in one Godot project; the only network traffic goes straight from the device to the public sources listed in the data-source register.

The stack is deliberately small: Godot owns UI, networking and rendering (no Flutter, no web view); GDScript is the only runtime language; Blender and GDAL are developer-machine tools whose output is committed to `assets/`. Godot's renderer targets Vulkan/Metal-class devices with Compatibility as the fallback.

Screens & user flow

`scripts/main.gd` is the screen manager. It owns the pipeline, the cache store and the settings, wires their signals to the screens, and shows exactly one screen at a time.

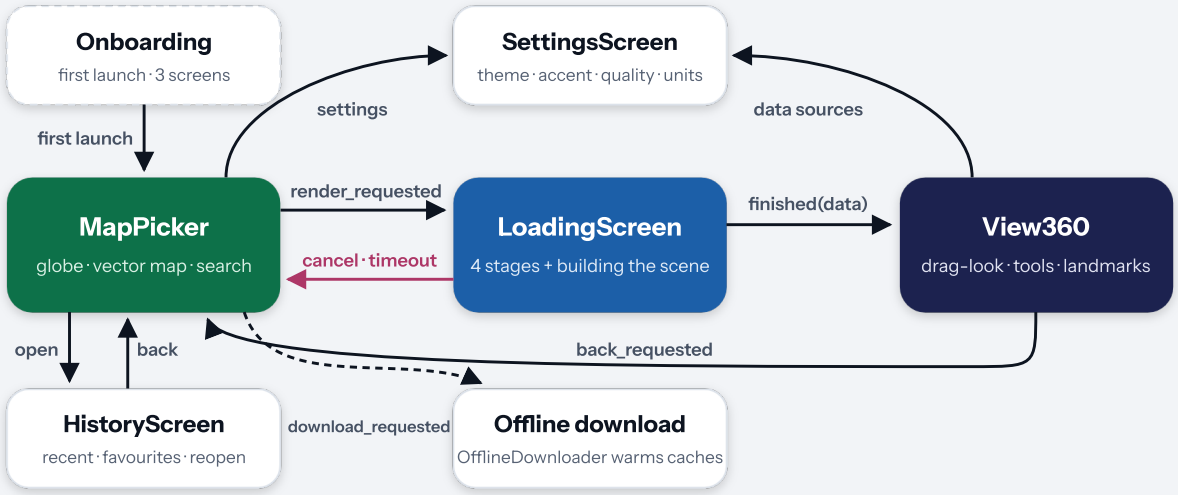
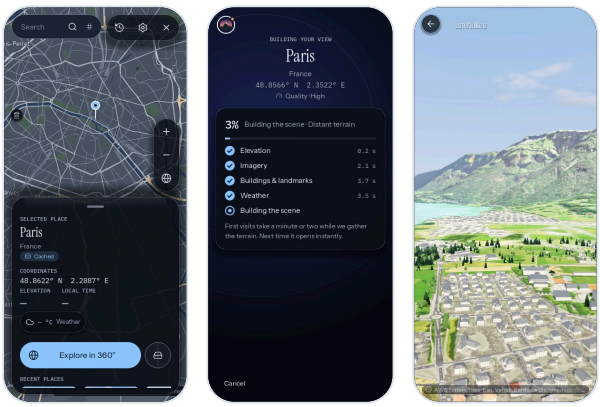


Figure 2. Screen state machine. A generation id is bumped on every start and cancel, so a world still building on its thread is dropped once superseded; starts are ignored while a run is in flight. A place that takes longer than 10 minutes is stopped and returns to the map with a plain message. Esc steps back one screen.

Interaction model

- **Map home** — a turning globe, then OpenFreeMap vector tiles drawn in the design system's Map style with Terrarium hillshade (`scripts/map/`); landmark markers from the registry and Wikipedia; OSM raster tiles as the fallback.
- **Search** — instant on-device predictions, then debounced Nominatim results with one request in flight; raw coordinates and pasted view links work too.
- **Loading** — four stage rows plus "Building the scene", driven by `stage_changed`, `stage_progress` and the build's phases; the terrain lens previews data as it lands; cancel returns within a second.
- **360 view** — drag to look, pinch/scroll zoom, sun-arc time and date, seasons, live weather, compass and callouts; tools: walk, time-lapse, horizon labels, mini-map, photo, view links, nearby landmarks.
- **Responsive** — one 390×844 pt design frame, `canvas_items` stretch, and phone / tablet / desktop breakpoints in `responsive.gd`.



Phone frame: map home, loading, 360 view (captured November 2026).

03 · DATA

Data sources

No source enters the fetch layer until its row in `docs/data-source-register.md` is complete: endpoint, format, licence, direct-mobile access, free-without-key, and a fallback. Every request goes through a fetcher module in `scripts/data/`.

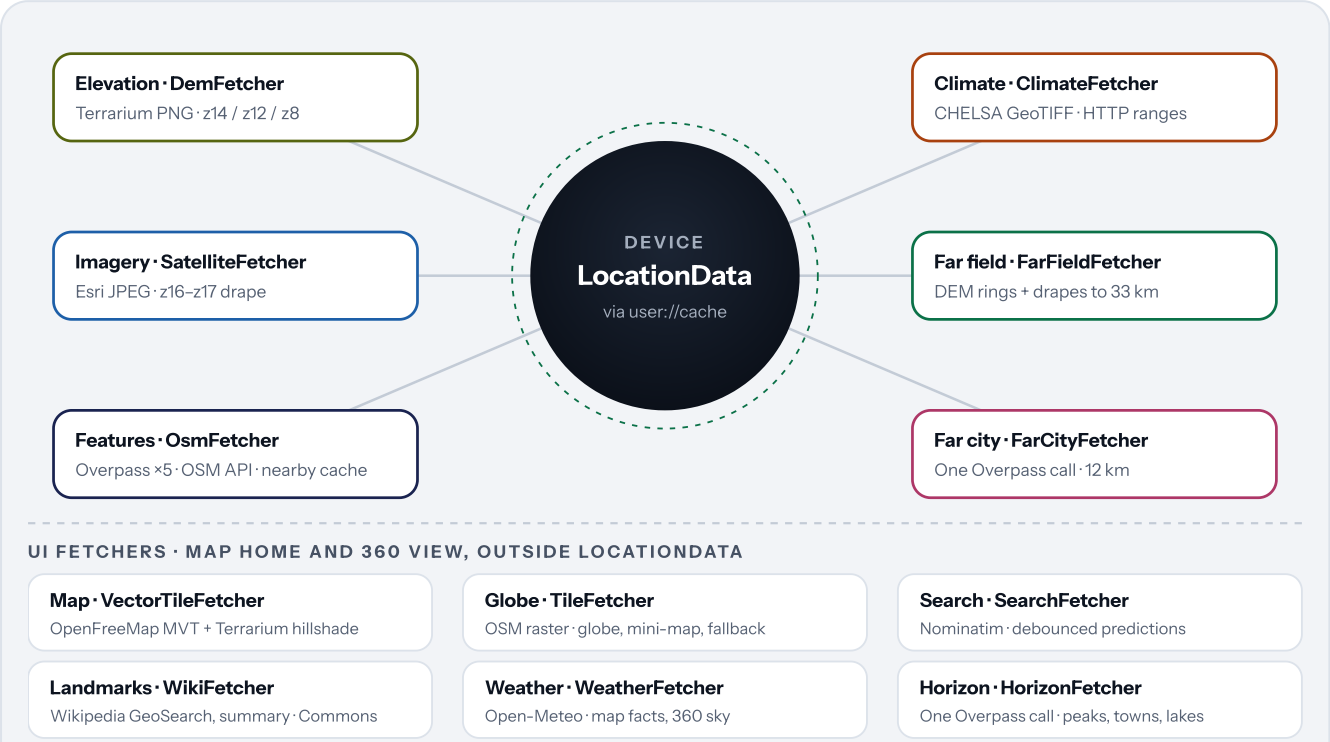


Figure 3. Fetch topology. World fetchers cache first and normalise into the `LocationData` contract, so generation never sees raw provider payloads. UI fetchers feed the map home and the 360 view directly, with their own caches.

NEED	SOURCE	FORMAT & ACCESS	LICENCE	FALLBACK
Elevation	AWS Terrain Tiles	PNG, h = R·256 + G + B/256 − 32768 m	Attribution (EU-DEM, USGS, SRTM…)	lower zoom → cache → procedural DEM
Imagery	Esri World Imagery	JPEG 256 px tiles, bounded pool	Esri ToU, noncommercial	lower zoom → cache → vertex colour
Buildings, roads, land	OSM Overpass · OSM API	JSON `out geom`, 5 instances, identifying UA	ODbL	cache → OSM API → nearby cache → absence-only
Climate	CHELSA V2.1	GeoTIFF row reads, ~0.3–0.6 MB/place	CC BY 4.0	latitude rules (ClimateModel)
Weather	Open-Meteo	JSON current conditions	CC BY	omit weather dressing
Search	Nominatim	JSON · debounced · 1 in flight	ODbL	gazetteer · coordinates
Map	OpenFreeMap	MVT + Terrarium hillshade	ODbL · OpenMapTiles	cache · OSM raster tiles
Landmarks	Wikipedia, Commons	GeoSearch · summary · photo	CC BY-SA · per photo	registry markers only
Textures	Poly Haven (build time)	19 PBR sets, 1k jpg in `assets/`	CC0	procedural textures

Generation pipeline

`GenerationPipeline` (`scripts/generation/pipeline.gd`) runs four stages, one per live data source as the loading screen names them. All network fetches start together in `run()` and overlap; each stage awaits only what it needs, with a bounded wait so a slow source degrades a layer instead of blocking the view.

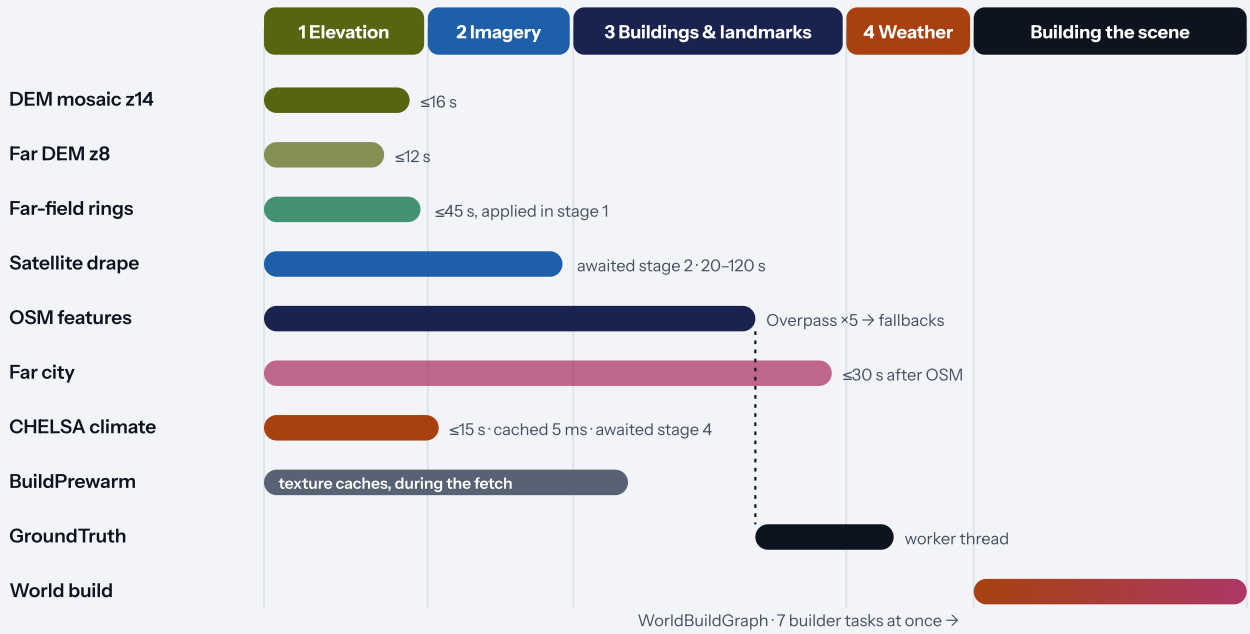
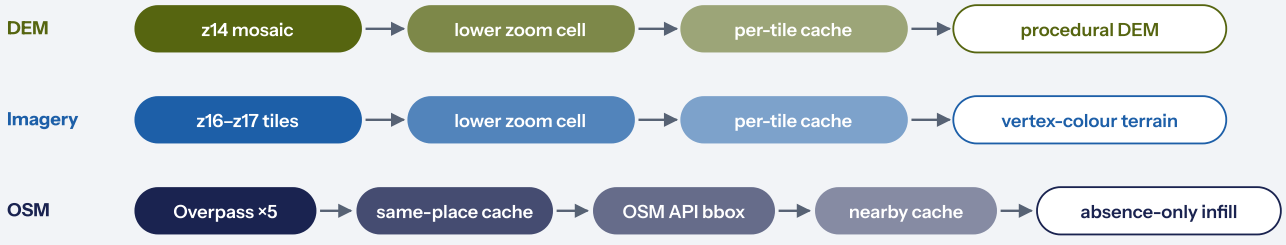


Figure 4. Overlapped fetch timeline (schematic). Stage rows are emitted as `stage_changed(i, label, "in_progress" | "done")`; `stage_progress` and `progress_changed` report real items done/total, never a timer. `GroundTruth` starts on a worker thread as soon as the OSM data is in, so it overlaps the far-city and climate waits.

Fallback chains

Fallbacks run only on absence of real data, in a fixed order per layer. They log with `print` (an expected outcome offline), so a permanently broken endpoint is found by grepping for *fetch failed*.



A cached payload is served before any of this; missing tags fall back to documented defaults inside the parser. After the last stage the pipeline waits for `BuildPrewarm`, then `View360` builds the world on a background thread through `WorldBuildGraph` ("Building the scene"). A cancel or the 10-minute timeout bumps the generation id in `main.gd` and asks a running build to stop at its next stage (`BuildAbort`), so a superseded world is freed rather than shown and never cached.

World model

The world-model contract (`docs/world-model-contract.md`) freezes three things: the `LocationData` fields, the pipeline stage order, and preset semantics. Changing any of them needs a contract task first.

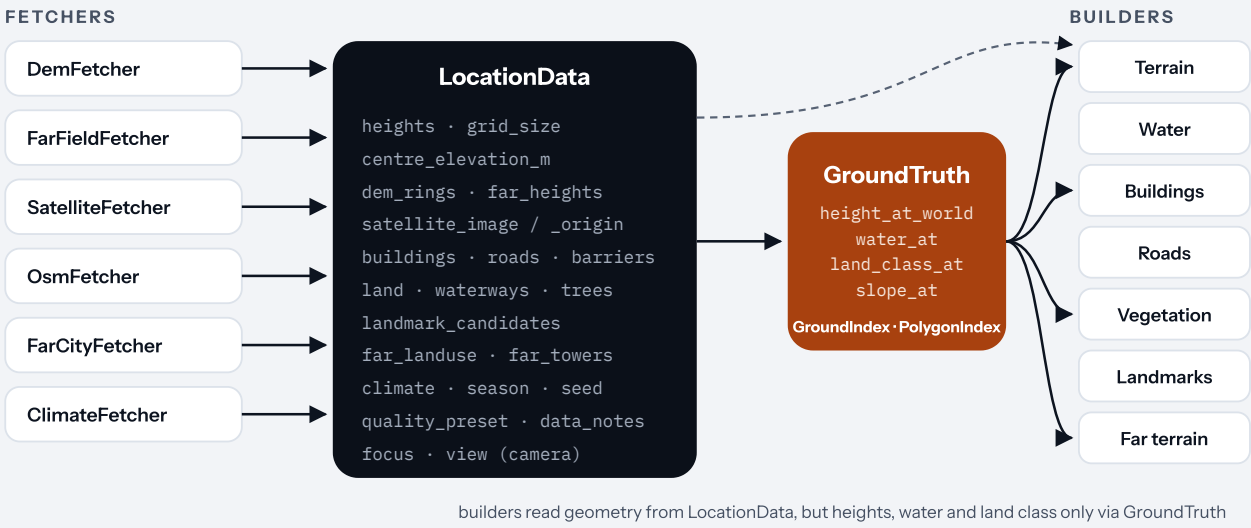
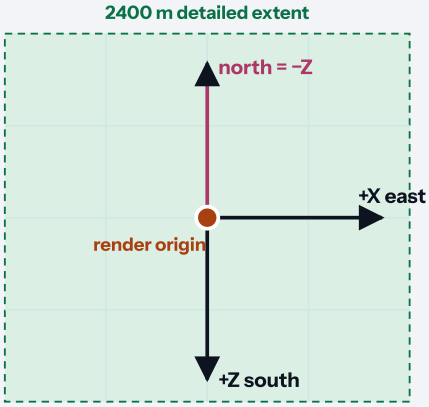


Figure 5. Data contract. Builders take `(data: LocationData, ground: GroundTruth)`, emit metre-space `Node3D` subtrees, and build materials only through `MaterialLibrary`.

Units and the render origin

`WorldScale` owns every conversion. One unit is one metre horizontally *and* vertically — the v1 220-unit world and adaptive vertical squash are gone. `+X` is east, `+Z` is south (north is `−Z`).

The detailed world is **2400 m** across (`DETAILED_EXTENT_M`). The render origin is set once per place at the viewpoint (lat, lon, `centre_elevation_m`), so the camera always works near the floating-point sweet spot.



1 unit = 1 m · Y = metres above origin

GroundTruth

Built once per place from DEM posts, OSM polygons and waterways, on a worker thread as soon as the OSM data is in: a smoothed surface flattened only where OSM says so (building pads, roads) with water carved in. `GroundIndex` and `PolygonIndex` grids answer land-class, building and point-in-polygon lookups exactly like the full scans they replace. All samplers are pure bilinear reads — outside the extent they clamp to the nearest edge and never return NaN.

06 · ASSEMBLY

World assembly

`TerrainBuilder.build_world(data, ground)` hands the scene to `WorldBuildGraph`: seven builder tasks run at once on `WorkerThreadPool`, each reading only `data` and `ground`, and their subtrees are adopted in a fixed order, so the parallel world matches the serial one byte for byte. Every layer samples the same `GroundTruth`, and vegetation masks built from the same data keep trees off roads, buildings and water.

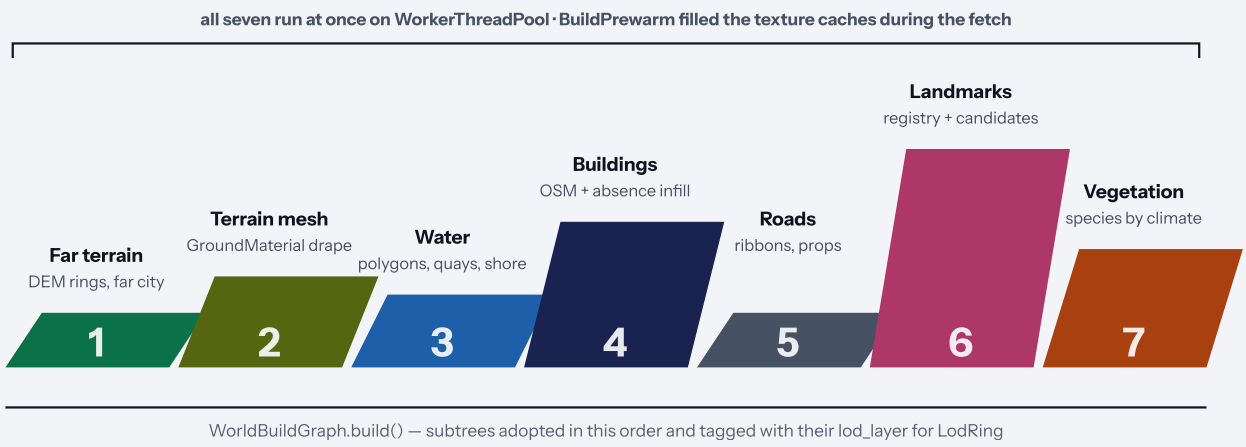
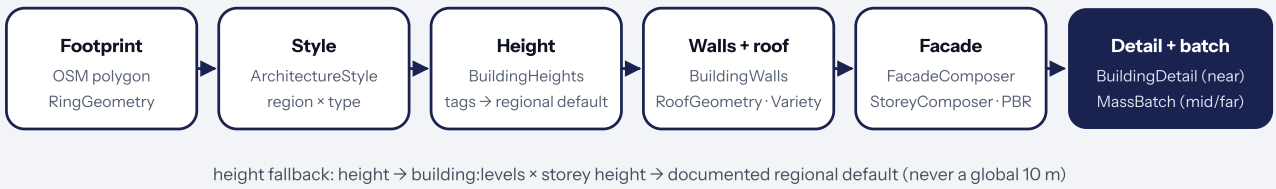


Figure 6. Adoption order. Each builder returns a detached contract subtree; `_adopt_flat` lifts its children into the root and records the layer they came from. A stop request (`BuildAbort`) returns an empty world; if the shared PBR material cache overflows, the build reruns serially for an exact order.

Building subsystem

Buildings are the largest subsystem (≈35 classes). Real OSM footprints are extruded first; procedural infill is **absence-only and style-matched** (`InfillPlanner`, `SettlementMap`) and never contradicts mapped data.



Terrain, water, roads

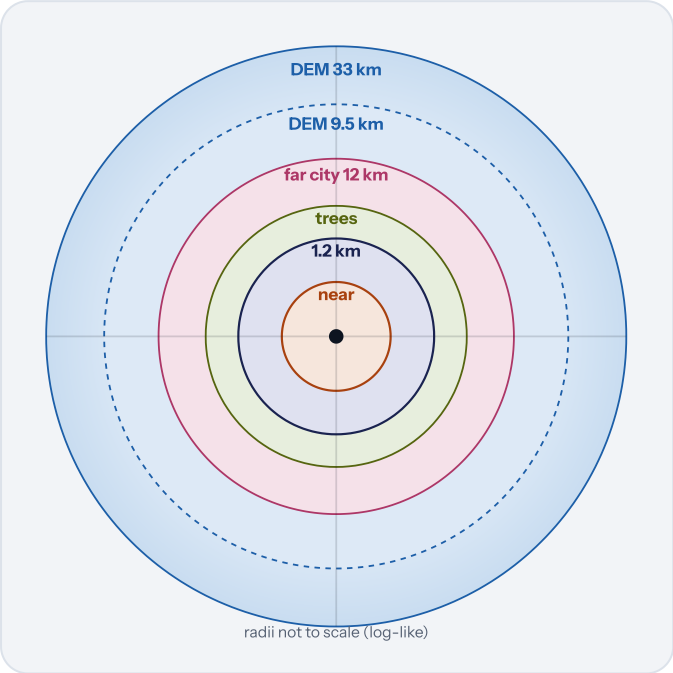
- **Terrain** — 96–256 mesh posts per side by preset over the 256-post `GroundTruth`; satellite drape and land-class tint in `GroundMaterial`'s shader.
- **Water** — triangulated OSM water, width-aware waterways, coastlines, shore aprons and stone quays where OSM implies built waterfront.
- **Roads** — attributed OSM roads as draped ribbon meshes; street furniture and barriers from `StreetProps`.

Vegetation & climate

- `ClimateModel` is the sole interpreter of CHELSA normals: snow line per season, treeline, biome and summer dryness.
- `TreeSpecies` picks a species mix from climate and land cover; `TreeMeshLibrary` builds procedural meshes, instanced near and impostors far.
- `VegetationMask` rasters keep trees and props off roads, buildings and water.

LOD & rendering

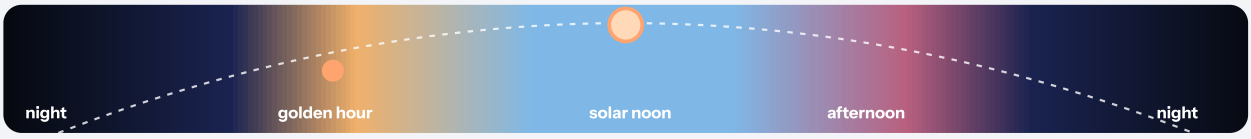
The 360 camera never translates, so level of detail is static: `LodRing` writes Godot visibility ranges once per build, with a 12% fade hysteresis. A preset switch only re-runs `apply()` — nothing is rebuilt, moved or recoloured.



RING	REACH	CONTENT
near	200–700 m	Full geometry with roofline detail (chimneys, dormers, rooftop plant)
mid	to 1.2 km	Building masses merged per material and 400 m cell; shared distant facades
trees	600–1700 m	Instanced species meshes, impostors past the cap
far city	4.5–12 km	Batched textured masses per octant × distance band, real towers ≥ 50 m
far DEM	to 33 km	~12.5 m/px to 4 km, ~25 m/px to 9.5 km, ~100 m/px to 33 km, with imagery drapes

Reach ranges span Fast → Ultra (`DETAIL_M`, `TREES_M`, `FAR_CITY_M` in `lod_ring.gd`). Draw calls scale with cells × materials, not with buildings.

Atmosphere, sun and weather



SolarPosition
NOAA, true sun + moon

AtmosphereRig
SkyPalette · SkyShader

WeatherLook
live weather · LightningRig

PostGrade
ACES grade · CityLights

ViewConditions
season, rain, snow, roofs

The view opens on the place's golden hour; the sun arc and date picker move the true solar position for the location and any calendar date, and the time-lapse plays the day. Live Open-Meteo weather drives cloud cover and precipitation through `WeatherLook`, with `LightningRig` strikes only in thunderstorms; `CityLights` lights windows at dusk. `RoofSnow` and `ClimateModel` lay snow on roofs and ground from the season and CHELSA snow days. `RenderQuality` adds the photoreal layer per tier (SSAO, volumetrics, window recesses on Ultra).

08 · QUALITY

Quality presets & governor

One user choice matters: the render preset. It trades accuracy for time — loading time paid once, frame time paid every frame. `GroundTruth` resolution is preset-independent, so switching presets never moves objects.

	FAST	BALANCED	HIGH	ULTRA
Fidelity level	1	2	2	3
Terrain mesh posts / side	96	128	192	256
Satellite zoom	z16	z16	z17	z17
Max real buildings	600	1 200	2 500	6 000
Trees (urban / rural)	70 / 90	120 / 170	170 / 230	380 / 500
Render scale	65%	85%	100%	125%
Shadows (distance)	off	2.4 km	2.4 km	4 km
Cloud octaves	3	4	5	5
First build / revisit	**≤20 s / ≤3 s**	**≤45 s / ≤5 s**	**≤75 s / ≤5 s**	**1–2 min / ≤8 s**

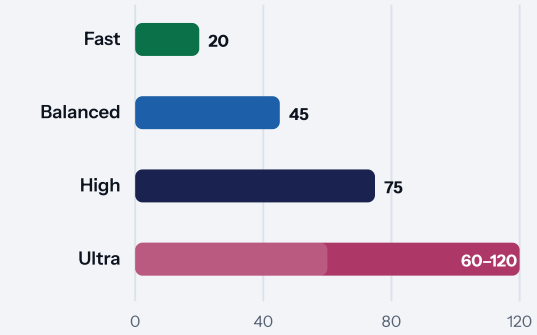


Figure 7. Broadband first-build budgets (plan §3.1). Cached revisits load the finished world instead.

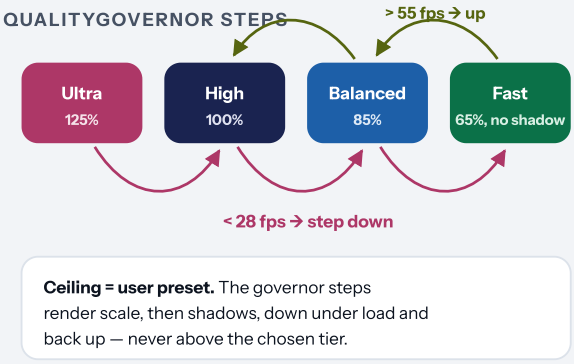


Figure 8. FPS-driven stepping. The HUD pill shows the tier plus the reduced render scale while stepping down.

09 · CACHE

Caching & offline

Two independent caches with separate budgets: the **data cache** holds fetched tiles and responses, the **world cache** holds finished worlds. A ~200 MB world can never evict the DEM, OSM or satellite tiles it was built from.

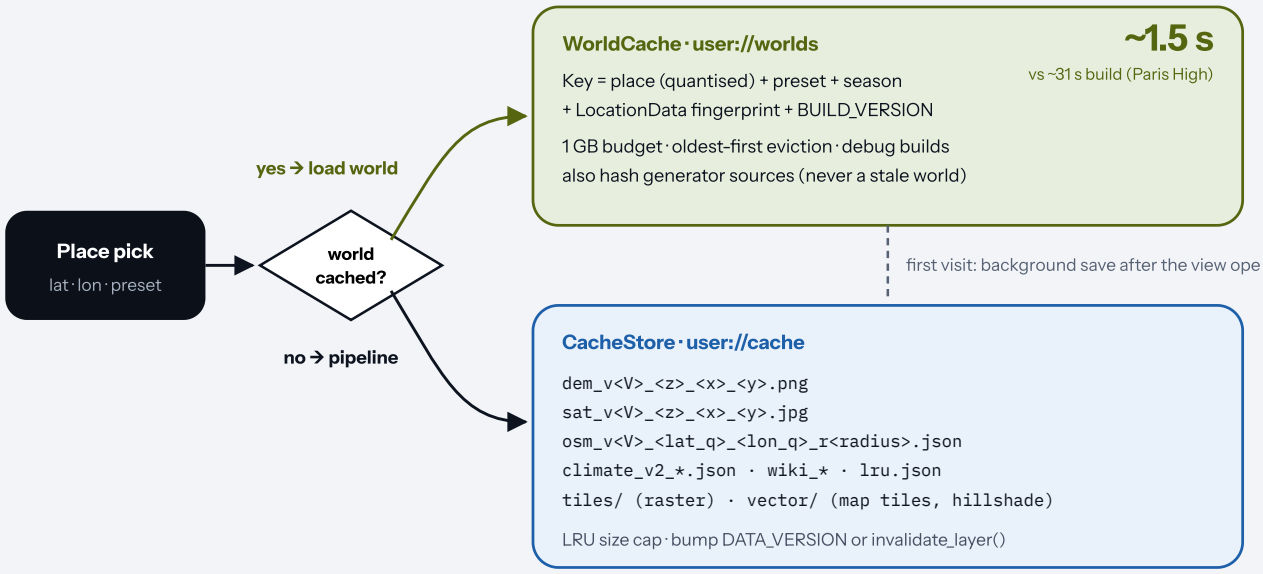


Figure 9. Revisit path. Headless test runs (dummy renderer) bypass the world cache entirely.

History

`CacheStore` is also the history store: a JSON file in `user://`. Generated terrain is reproducible from the stored coordinates and seed, so history entries double as cache keys and carry their v2 key set. Each entry also keeps the camera as you left it (`ViewLink`) and a picture of the view (`PlaceThumb`, `user://thumbs`).

On macOS `user://` is `~/Library/Application Support/Planet Render 360`; `UserDirMigration` moves data from Godot's default directory on the first launch after the switch.

Versioning

Legacy v1 keys (`dem_`, `sat_`, `osm3_`) are ignored, never migrated; the LRU reaps them first under cap pressure. Area keys quantise coordinates to `round(deg × 1000)`.

Offline download

"Download for offline" in the picker runs `OfflineDownloader`, which warms every on-device cache for a place — DEM, imagery, OSM, far field and climate — with progress in the picker and a history entry on success.

Offline gate: with networking disabled, a cached place loads in seconds and renders identically. An uncached place offline still renders, falling back to the deterministic procedural DEM.

Landmarks

Recognisable structures come from three paths: a **curated registry** modelled on the fly in true metres, **well-mapped OSM landmarks** drawn from their own building parts, and **archetypes** (church, tower, dome, stadium, wheel…) for any other notable OSM feature. Wikipedia supplies the markers and details people tap.



Figure 10. Curated registry (`LocationData`): 23 landmarks in 18 builder classes, listed without their `Landmark` prefix. Inside an entry's 2–13 km trigger radius the landmark stands at its true bearing and distance, beside any other registry sites in range.

LANDMARK	CLASS	HEIGHT
Eiffel Tower	`Eiffel`	330 m
Tokyo Tower	`Tokyo`	333 m
Big Ben	`Westminster`	96 m
Colosseum	`Colosseum`	48 m
Sagrada Família	`Sagrada`	172 m
Giza · Liberty	`Monuments`	139 · 93 m
Sydney Opera House	`Opera`	65 m
Helsinki · Turku	`Cathedrals`	62 · 101 m
Burj Khalifa	`Burj`	828 m

LANDMARK	CLASS	HEIGHT
One WTC · Empire State	`NycTowers`	541 · 443 m
Salute, Venice	`Salute`	60 m
Monte Carlo Casino	`Casino`	34 m
Gwanghwamun	`Gwanghwamun`	19 m
Banff Springs Hotel	`Banff`	60 m
Stockholm City Hall	`Stockholm`	106 m
Golden Gate · Brooklyn	`SuspensionBridges`	227 · 84 m
Tower Bridge	`TowerBridge`	65 m
Sydney Harbour · Bixby	`ArchBridges`	134 · 79 m

How a landmark is made

- `LandmarkMesh` merges geometry, `LandmarkMaterials` supplies calibrated PBR; models are built on device (`assets/eiffel.glb` proves the Blender path).
- `LandmarkSites` picks what a place builds: registry sites in range, then ranked OSM features (6–10 by preset) inside the 9 km haze limit.
- `LandmarkFidelity` keeps a well-mapped landmark's real building and its `building:part` domes; the rest become `LandmarkArchetypes`.
- `WikiFetcher` markers and the `LandmarkCard` on the map and in the view; `LandmarkVisit` picks the standoff for Go there.

Code organisation & testing

Every class is one GDScript file under `scripts/` and stays at or under **1000 lines**; `tools/check_class_size.sh` fails the build on any violation. Large concerns split into focused helpers (for example `building_builder` → walls, heights, detail, roofs; `view_360` → chrome, conditions, tools, walk).



Package responsibilities

`data/`	Fetchers, parsers, contract, cache, history
`generation/`	Pipeline, ground surface, every builder, materials
`render/`	Camera, atmosphere, LOD, governor, presets, world cache
`screens/`	Onboarding, map home, loading, 360 view and its tools, history, settings
`map/`	Vector map: MVT decode, Map style, hillshade, labels
`scripts/*.gd`	Screen manager, theme, prefs, icons, responsive rules, user-dir migration

Conventions

- Each class opens with a `##` doc comment stating its responsibility and the contract or plan task it implements.
- Static builders with `(data, ground)` signatures; heavy mesh work on worker threads via `MeshArrays` and `WorldBuildGraph`.

Test suite (52 scripts)

AREA	TESTS
Contracts & flow	smoke, flow, class_registry, build_guard, leak, shutdown_guard, generation_timeout, data_notes, user_dir
Data & fetch	osm, osm_fallback, dem, tiles, search, weather, climate
World & scale	world_scale, terrain_scale, ground_index, polygon_index, scenery, arid_ground, building_fabric, architecture, facade_relief, far_city, wheel_ground, lod, parallel_build
Sky & light	sky, solar (vs ephemeris), lightning, accent_sun, render_upgrade
UI & map	responsive, zoom_search, map_pan, progressive_map, vector_map, exit_tile, settings, loading_building
Landmarks & viewer	landmark, landmark_fidelity, landmark_info, landmark_list, monument, preset_views, viewer_tools
Windowed · live	screenshot, world_cache · fidelity_scorecard

Run any test with `godot --headless --script tests/<name>.gd`; windowed tests drop `--headless` and on macOS run through `tools/godot_bg.sh`. The fidelity scorecard (`docs/fidelity-scorecard.md`) checks golden locations — Paris, Interlaken, Banff, Big Sur, Turku.

12 · GALLERY

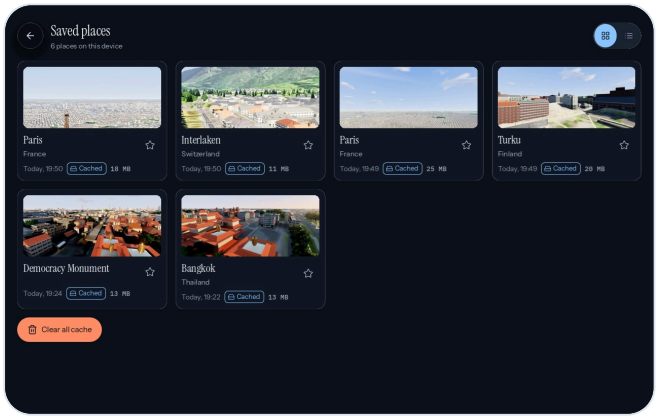
Gallery & attribution

Captures from the current build (`tests/screenshot_test.gd`, November 2026), High preset, desktop frame.



Viewer tools, Interlaken — horizon labels, mini-map, the date sheet and the sun arc over the 360 view.





Left: Interlaken valley — z14 DEM, far-field rings to the horizon, climate-driven vegetation. **Right:** saved places with their pictures.

Attribution

Author: **Petri Tuononen** | https://www.linkedin.com/in/petrituononen

© OpenStreetMap contributors (ODbL) · Elevation: AWS Terrain Tiles / Tilezen (EU-DEM / Copernicus, USGS 3DEP, SRTM and others) · Imagery: Esri, Vantor, Earthstar Geographics and the GIS User Community · Climate: CHELSA V2.1 (Karger et al., 2017, CC BY 4.0) · Weather: Open-Meteo (CC BY) · Search: Nominatim · Map: OpenFreeMap © OpenMapTiles · Landmark text: Wikipedia (CC BY-SA 4.0) · Photos: Wikimedia Commons, credited per photo · Coastlines: Natural Earth (public domain) · Textures: Poly Haven (CC0) · Foliage textures: own procedural work (CC0). Code distributed under the MIT License.